

Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées Al Hoceima

COMPTE RENDU

TP N°04 : Projet JEE – MVC1 (Servlet, JSP & JSTL)

Pr. Mohamed CHERRADI

Ouhadi Chaimae

Filière : TDIA2 - S4

1. Introduction:

Ce projet consiste à développer une application de Gestion des Produits permettant de réaliser les opérations CRUD (Create, Read, Update, Delete) sur des produits, en utilisant les technologies fondamentales du développement web Java EE, à savoir les Servlets, les pages JSP et la bibliothèque de balises JSTL.

En plus, l'application est enrichie avec un mécanisme d'authentification basé sur les sessions HTTP, ainsi qu'une gestion des erreurs.

2. Objectifs du projet

Ce projet vise :

- La maîtrise de l'architecture **MVC1** (Model – View – Controller) appliquée au développement web Java EE.
- La compréhension du rôle et du fonctionnement des Servlets.
- L'utilisation des pages JSP et de la bibliothèque JSTL.
- L'application de **l'architecture 3-tiers** : couche DAO, couche Service (Métier) et couche Web.
- L'implémentation d'un **mécanisme d'authentification** par session HTTP.
- La gestion **des erreurs et des exceptions** au niveau des Servlets.
- La configuration correcte du descripteur de déploiement **web.xml**.

3. Architecture du projet

Architecture MVC1 et 3-tiers

Le projet adopte **une architecture MVC1** combinée à **une architecture 3-tiers**. La particularité de MVC1 est que chaque requête HTTP est traitée par une Servlet dédiée, contrairement à MVC2 où une seule Servlet frontale dispatche toutes les requêtes.

Le flux de traitement d'une requête suit le schéma suivant :

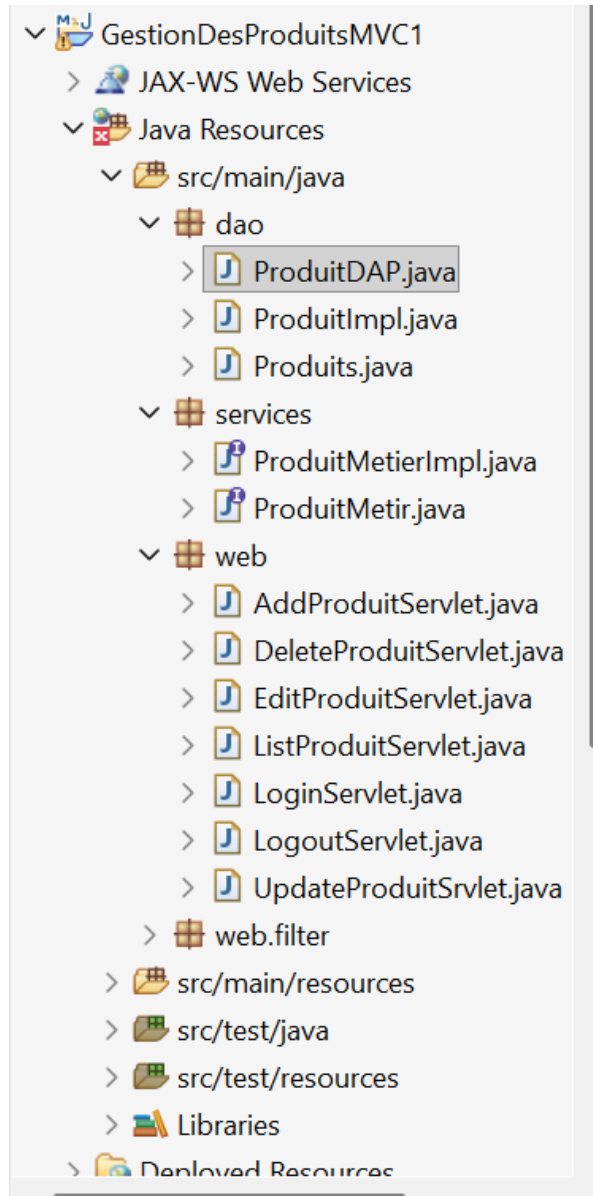
Navigateur → Servlet (Controller) → Service (Métier) → DAO → JSP (View)

Les trois couches:

- **Couche DAO:** Représentation et accès aux données.
- **Couche Services:** Logique métier, c'est l'intermédiaire entre les Controllers et le DAO.
- **Couche Web:** Réception des requêtes HTTP et navigation vers les files JSP.
- **Views:** Affichage dynamique avec JSTL (c:forEach, c:if, c:choose).

4. Structure du projet

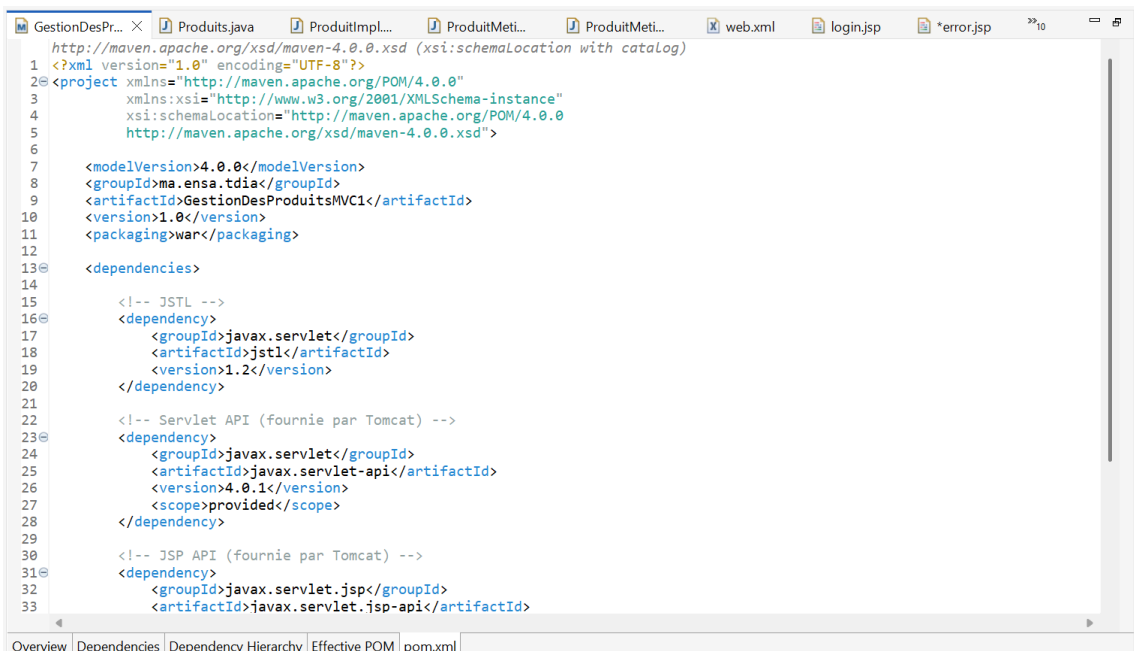
Le projet est organisé selon la structure standard Maven pour un projet web Java EE comme le montre la capture d'écran ci-dessous:



5. Implémentation

5.1 pom.xml

Le fichier [pom.xml](#) déclare les dépendances nécessaires au projet. Le packaging war indique à Maven de générer un fichier WAR déployable sur Tomcat.



5.2 Couche Model

Classe Produit

La classe Produit est le modèle de données de l'application. Elle représente l'entité manipulée par toutes les couches de l'architecture.

Deux constructeurs sont définis : un constructeur vide requis par les JSP pour créer des objets dynamiquement, et un constructeur paramétré utilisé lors de l'ajout d'un nouveau produit sans préciser l'ID.

```

1 package dao;
2
3 public class Produits {
4
5     private Long idProduit;
6     private String nom;
7     private String description;
8     private Double prix;
9
10
11     public Produits() {}
12
13
14     public Produits(String nom, String description, Double prix) {
15         this.nom = nom;
16         this.description = description;
17         this.prix = prix;
18     }
19
20     public Long getIdProduit() { return idProduit; }
21     public void setIdProduit(Long idProduit) { this.idProduit = idProduit; }
22
23     public String getNom() { return nom; }
24     public void setNom(String nom) { this.nom = nom; }
25
26     public String getDescription() { return description; }
27     public void setDescription(String description) { this.description = description; }
28
29     public Double getPrix() { return prix; }
30     public void setPrix(Double prix) { this.prix = prix; }
31 }

```

5.3 Couche DAO

Interface ProduitDAO

L'interface ProduitDAO définit le contrat des opérations CRUD.

```

1 package dao;
2
3 import java.util.List;
4
5 public interface ProduitDAO {
6
7     void addProduit(Produits p);
8     void deleteProduit(Long id);
9     Produits getProduitById(Long id);
10    List<Produits> getAllProduits();
11    void updateProduit(Produits p);
12 }

```

Classe ProduitImpl

La classe ProduitImpl implémente ProduitDAO en utilisant une ArrayList en mémoire comme base de données simulée. Un compteur d'ID garantit l'unicité des identifiants.

La méthode `init()` précharge quelques produits de test au démarrage. Elle est appelée par la couche Service via le constructeur du Singleton.

```

package dao;

import java.util.ArrayList;
import java.util.List;

public class ProduitImpl implements ProduitDAO {
    private List<Produits> produits = new ArrayList<>();
    private Long compteurId = 0L;

    public void init() {
        addProduit(new Produits("PC 1", "Sony Vaio 1", 7000.0));
        addProduit(new Produits("PC 2", "Sony Vaio 2", 6000.0));
        addProduit(new Produits("PC 3", "Sony Vaio 3", 4000.0));
    }

    @Override
    public void addProduit(Produits p) {
        compteurId++;
        p.setIdProduit(compteurId);
        produits.add(p);
    }

    @Override
    public void deleteProduit(Long id) {
        Produits aSupprimer = getProduitById(id);
        if (aSupprimer != null) {
            produits.remove(aSupprimer);
        }
    }

    @Override
    public Produits getProduitById(Long id) {
        for (Produits p : produits) {
            if (p.getIdProduit().equals(id)) {
                return p;
            }
        }
        return null;
    }
}

```

5.4 Couche Service

Interface ProduitMetier

L'interface ProduitMetier sert d'intermédiaire entre les Servlets et le DAO. Elle peut contenir de la logique métier supplémentaire (validation, règles métier...).

```

1 package services;
2
3 import java.util.List;
4 import dao.Produits;
5
6 public interface ProduitMetier {
7
8     void addProduit(Produits p);
9     void deleteProduit(Long id);
10    Produits getProduitById(Long id);
11    List<Produits> getAllProduits();
12    void updateProduit(Produits p);
13 }
```

Classe ProduitMetierImpl: Pattern Singleton

La classe ProduitMetierImpl implémente le design pattern Singleton pour garantir qu'une seule instance est partagée entre toutes les Servlets. Sans ce pattern, chaque Servlet créerait sa propre liste de produits indépendante.

Toutes les Servlets appellent `ProduitMetierImpl.getInstance()`. Elles partagent ainsi la même ArrayList de produits en mémoire. Sans Singleton, ajouter un produit dans une Servlet ne serait pas visible dans une autre.

```

1 package services;
2
3 import java.util.List;
4 import dao.Produits;
5 import dao.ProduitDAP;
6 import dao.ProduitImpl;
7
8 public class ProduitMetierImpl implements ProduitMetier {
9     private static ProduitMetierImpl instance;
10    private ProduitDAP dao;
11
12    private ProduitMetierImpl() {
13        dao = new ProduitImpl();
14        ((ProduitImpl) dao).init();
15    }
16
17    public static ProduitMetierImpl getInstance() {
18        if (instance == null) {
19            instance = new ProduitMetierImpl();
20        }
21        return instance;
22    }
23
24    @Override
25    public void addProduit(Produits p) { dao.addProduit(p); }
26
27    @Override
28    public void deleteProduit(Long id) { dao.deleteProduit(id); }
29
30    @Override
31    public Produits getProduitById(Long id) { return dao.getProduitById(id); }
32
33    @Override
34    public List<Produits> getAllProduits() { return dao.getAllProduits(); }
35
36    @Override
```

5.5 Couche Web: Les Servlets

Conformément à l'architecture MVC1, chaque opération CRUD est prise en charge par une Servlet dédiée. Toutes les Servlets héritent de HttpServlet et utilisent le Singleton de la couche Service.

ListProduitServlet: Affichage et Recherche

Cette Servlet répond aux requêtes GET. Elle gère deux cas : afficher tous les produits, ou filtrer par ID si le paramètre idProduit est fourni.

J'ai utilisé forward() pour transmettre les attributs de la requête à la JSP. Avec sendRedirect(), les attributs seraient perdus car une nouvelle requête HTTP serait déclenchée.

```
package web;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import dao.Produits;
import services.ProduitMetir;
import services.ProduitMetierImpl;

public class ListProduitServlet extends HttpServlet {
    private static final ProduitMetir metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String idParam = req.getParameter("idProduit");
        List<Produits> liste = new ArrayList<>();

        try {
            if (idParam != null && !idParam.trim().isEmpty()) {
                Long id = Long.parseLong(idParam.trim());
                Produits p = metier.getProduitById(id);
                if (p != null) {
                    liste.add(p);
                } else {
                    req.setAttribute("messageInfo", "Aucun produit trouvé avec l'ID : " + id);
                    liste = metier.getAllProduits();
                }
            } else {
                liste = metier.getAllProduits();
            }
        }
    }
}
```

AddProduitServlet: Ajout

Cette Servlet répond aux requêtes POST du formulaire d'ajout. Elle récupère les paramètres, crée un objet Produit et l'ajoute via la couche Service.

```
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import dao.Produits;
import services.ProduitMetier;
import services.ProduitMetierImpl;

public class AddProduitServlet extends HttpServlet {

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String nom = req.getParameter("nom");
        String description = req.getParameter("description");
        String prixStr = req.getParameter("prix");

        if (nom == null || nom.trim().isEmpty()
            || description == null || description.trim().isEmpty()
            || prixStr == null || prixStr.trim().isEmpty()) {

            req.setAttribute("messageErreur", "Tous les champs sont obligatoires.");
            req.setAttribute("listeProduits", metier.getAllProduits());
            req.getRequestDispatcher("index.jsp").forward(req, resp);
            return;
        }

        try {
            Double prix = Double.parseDouble(prixStr.trim());
            // ... (code pour ajouter le produit) ...
        } catch (NumberFormatException e) {
            // ... (code pour gérer l'exception) ...
        }
    }
}
```

DeleteProduitServlet: Suppression

La suppression est déclenchée par un lien HTML `<a href='deleteProduit?id=...'>`. La Servlet utilise `sendRedirect()` après suppression pour éviter la resoumission du formulaire.

```
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import services.ProduitMetier;
import services.ProduitMetierImpl;

public class DeleteProduitServlet extends HttpServlet {

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String idParam = req.getParameter("id");

        try {
            Long id = Long.parseLong(idParam);

            if (metier.getProduitById(id) == null) {
                resp.sendRedirect("listProduits?erreur=Produit+introuvable");
                return;
            }

            metier.deleteProduit(id);
            resp.sendRedirect("listProduits");

        } catch (NumberFormatException e) {
            resp.sendRedirect("listProduits?erreur=ID+invalid");
        }
    }
}
```


EditProduitServlet + UpdateProduitServlet: Modification

La modification nécessite deux Servlets. `EditProduitServlet` charge le produit et le passe à la JSP via l'attribut `produitEdit` pour pré-remplir le formulaire. `UpdateProduitServlet` reçoit les données modifiées via POST et applique la mise à jour.

```
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import dao.Produits;
import services.ProduitMetier;
import services.ProduitMetierImpl;

public class EditProduitServlet extends HttpServlet {

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String idParam = req.getParameter("id");

        try {
            Long id = Long.parseLong(idParam);
            Produits p = metier.getProduitById(id);

            if (p == null) {
                req.setAttribute("messageErreur", "Produit introuvable.");
                req.setAttribute("listeProduits", metier.getAllProduits());
                req.getRequestDispatcher("index.jsp").forward(req, resp);
                return;
            }

            req.setAttribute("produitEdit", p);
            req.setAttribute("listeProduits", metier.getAllProduits());
            req.getRequestDispatcher("index.jsp").forward(req, resp);
        } catch (Exception e) {
            // Gestion des exceptions
        }
    }
}
```

```
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import dao.Produits;
import services.ProduitMetier;
import services.ProduitMetierImpl;

public class UpdateProduitServlet extends HttpServlet {

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String idStr = req.getParameter("idProduit");
        String nom = req.getParameter("nom");
        String description = req.getParameter("description");
        String prixStr = req.getParameter("prix");

        if (nom == null || nom.trim().isEmpty() ||
            description == null || description.trim().isEmpty() ||
            prixStr == null || prixStr.trim().isEmpty()) {

            req.setAttribute("messageErreur", "Tous les champs sont obligatoires.");
            req.setAttribute("listeProduits", metier.getAllProduits());
            req.getRequestDispatcher("index.jsp").forward(req, resp);
            return;
        }

        // Logique de mise à jour du produit
    }
}
```

5.6 Authentification

L'authentification a été ajoutée afin de sécuriser l'accès à l'application. Elle repose sur les sessions HTTP (HttpSession) et un filtre de Servlet.

LoginServlet

Cette Servlet gère à la fois l'affichage du formulaire (GET) et la vérification des identifiants (POST).

```
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class LoginServlet extends HttpServlet {

    private static final String LOGIN = "admin";
    private static final String PASSWORD = "admin123";

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        req.getRequestDispatcher("login.jsp").forward(req, resp);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String login    = req.getParameter("login");
        String password = req.getParameter("password");

        if (LOGIN.equals(login) && PASSWORD.equals(password)) {
            HttpSession session = req.getSession();
            session.setAttribute("utilisateur", login);
            resp.sendRedirect("listProduits");
        } else {
            req.setAttribute("erreurLogin", "Identifiants incorrects. Veuillez réessayer.");
        }
    }
}

package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class LogoutServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        HttpSession session = req.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        resp.sendRedirect("login");
    }
}
```

AuthFilter: Filtre de protection

Le filtre AuthFilter intercepte toutes les requêtes (/*) et vérifie la présence de l'utilisateur en session. Il laisse passer uniquement les requêtes vers /login et /logout sans authentification.

Sans ce filtre, un utilisateur pourrait accéder directement à /listProduits sans être connecté. Grâce au filtre déclaré dans web.xml avec le pattern /*, toutes les pages sont protégées automatiquement.

```

1 package web.filter;
2
3 import java.io.IOException;
4 import javax.servlet.Filter;
5 import javax.servlet.FilterChain;
6 import javax.servlet.FilterConfig;
7 import javax.servlet.ServletException;
8 import javax.servlet.ServletRequest;
9 import javax.servlet.ServletResponse;
10 import javax.servlet.http.HttpServletRequest;
11 import javax.servlet.http.HttpServletResponse;
12 import javax.servlet.http.HttpSession;
13
14 public class AuthFilter implements Filter {
15     @Override
16     public void init(FilterConfig filterConfig) throws ServletException {}
17
18     @Override
19     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
20         throws IOException, ServletException {
21         HttpServletRequest req = (HttpServletRequest) request;
22         HttpServletResponse resp = (HttpServletResponse) response;
23
24         String uri = req.getRequestURI();
25         if (uri.endsWith("/login") || uri.endsWith("/logout")) {
26             chain.doFilter(request, response);
27             return;
28         }
29         HttpSession session = req.getSession(false);
30         boolean connecte = (session != null && session.getAttribute("utilisateur") != null);
31
32         if (connecte) {
33             chain.doFilter(request, response);
34         }
35     }
36 }

```

5.7 web.xml: Le descripteur de déploiement

Le fichier web.xml est le fichier de configuration central de l'application. Il déclare toutes les Servlets avec leurs URL mappings, le filtre d'authentification et les pages d'erreur personnalisées.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xmlns="http://java.sun.com/xml/ns/javaee"
4   xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
5   http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
6   id="WebApp_ID" version="3.0">
7
8   <display-name>GestionDesProduitsMVC1</display-name>
9
10  <welcome-file-list>
11    <welcome-file>listProduits</welcome-file>
12  </welcome-file-list>
13
14  <filter>
15    <filter-name>authFilter</filter-name>
16    <filter-class>web.filter.AuthFilter</filter-class>
17  </filter>
18  <filter-mapping>
19    <filter-name>authFilter</filter-name>
20    <url-pattern>*/</url-pattern>
21  </filter-mapping>
22
23  <servlet>
24    <servlet-name>login</servlet-name>
25    <servlet-class>web.LoginServlet</servlet-class>
26  </servlet>
27  <servlet-mapping>
28    <servlet-name>login</servlet-name>
29    <url-pattern>/login</url-pattern>
30  </servlet-mapping>
31
32  <servlet>
33    <servlet-name>logout</servlet-name>
34    <servlet-class>web.LogoutServlet</servlet-class>

```

6. Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux du développement web Java EE dans le cadre d'une architecture MVC1 structurée et maintenable.

Sur le plan technique, nous avons appris à :

- Organiser un projet en couches séparées (DAO, Service, Web, View) selon le principe de séparation des responsabilités.
- Implémenter le design pattern Singleton pour partager une même instance entre plusieurs Servlets et maintenir un état cohérent en mémoire.
- Utiliser les Expression Language (EL) et les balises JSTL pour créer des vues JSP dynamiques et lisibles, sans code Java dans la présentation.
- Configurer le descripteur de déploiement web.xml pour mapper les URLs aux Servlets et déclarer les filtres et les pages d'erreur.
- Implémenter un mécanisme d'authentification par session HTTP, en protégeant l'ensemble des ressources via un filtre de Servlet.

Sur le plan architectural, ce projet illustre clairement la différence entre MVC1 (une Servlet par opération) et MVC2 (une Servlet frontale unique). MVC1, bien que plus verbeux, offre une structure simple et directe.

Enfin, l'utilisation de Maven comme outil de gestion de dépendances nous a permis de nous affranchir de la gestion manuelle des fichiers JAR et d'adopter une approche de développement plus professionnelle et reproductible.